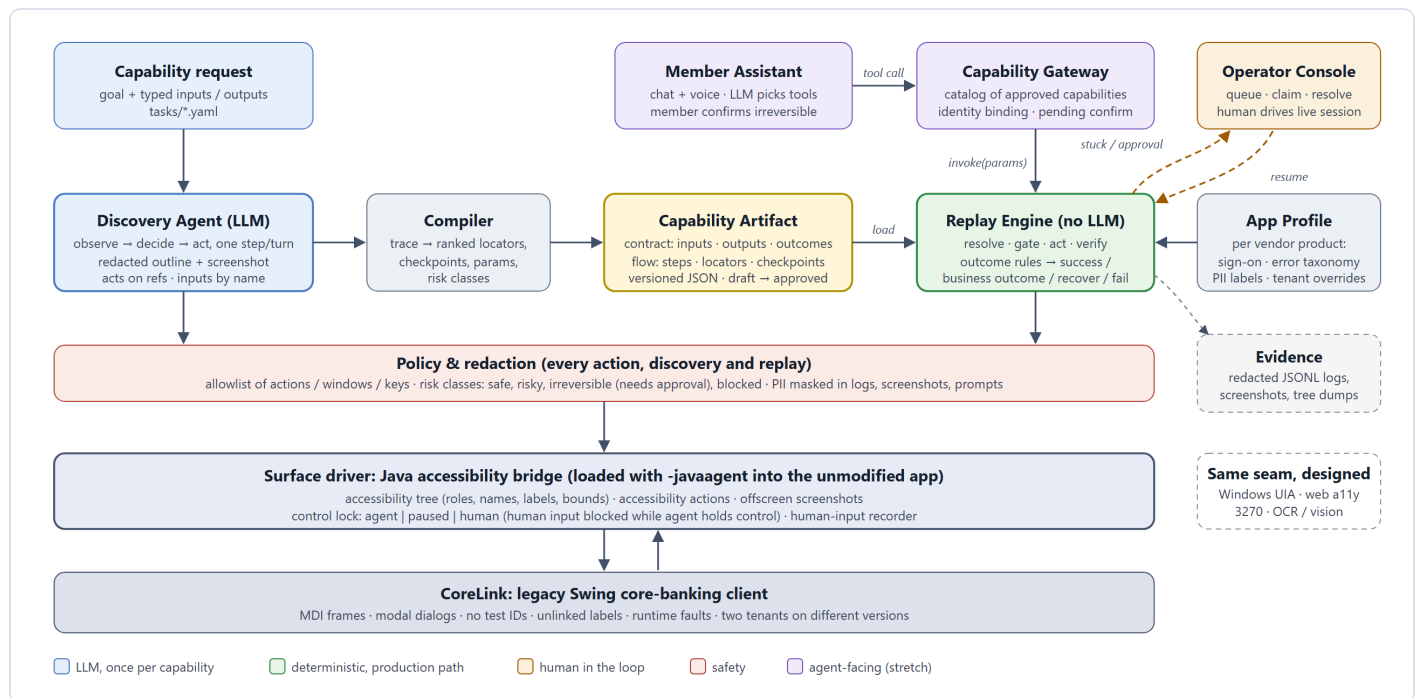


InterfaceAgent

An LLM discovers how to do a task in a legacy banking app once. The run becomes a typed, versioned capability that replays deterministically, classifies runtime errors, stays inside safety policy, and hands the live session to a human when it must.

Karol Stuart · September 26, 2026

Repository: <https://github.com/karolgit/InterfaceAgent>



Summary

The target is **CoreLink**, a legacy-style Java Swing core-banking desktop client built as the proxy app. It has no DOM and no test IDs, its labels aren't linked to their fields, it uses internal frames and modal dialogs, and faults can be injected at runtime. All data is fake. Four pieces carry the design:

- **A surface driver.** A Java agent reads the unmodified app's accessibility tree, acts through accessibility actions, and owns the human-handoff control lock.
- **A discovery agent.** A pluggable LLM planner (Claude or Gemini) acts on element references and never sees raw PII.
- **A capability artifact.** It has a contract part for calling agents and a flow part for replay.
- **A replay engine.** It sorts every run into a business outcome, a recovered condition, or a hard failure.

The thread runs end to end: goal, LLM run, saved capability, deterministic replay with parameters, outputs, and error handling, human handoff on the live session, and evidence for each. Every core requirement in the brief is covered, as the matrix below shows. The stretch goals built are the agent-facing capability interface and cross-tenant reuse with per-tenant overrides. Unattended replay is also gated on draft or approved status.

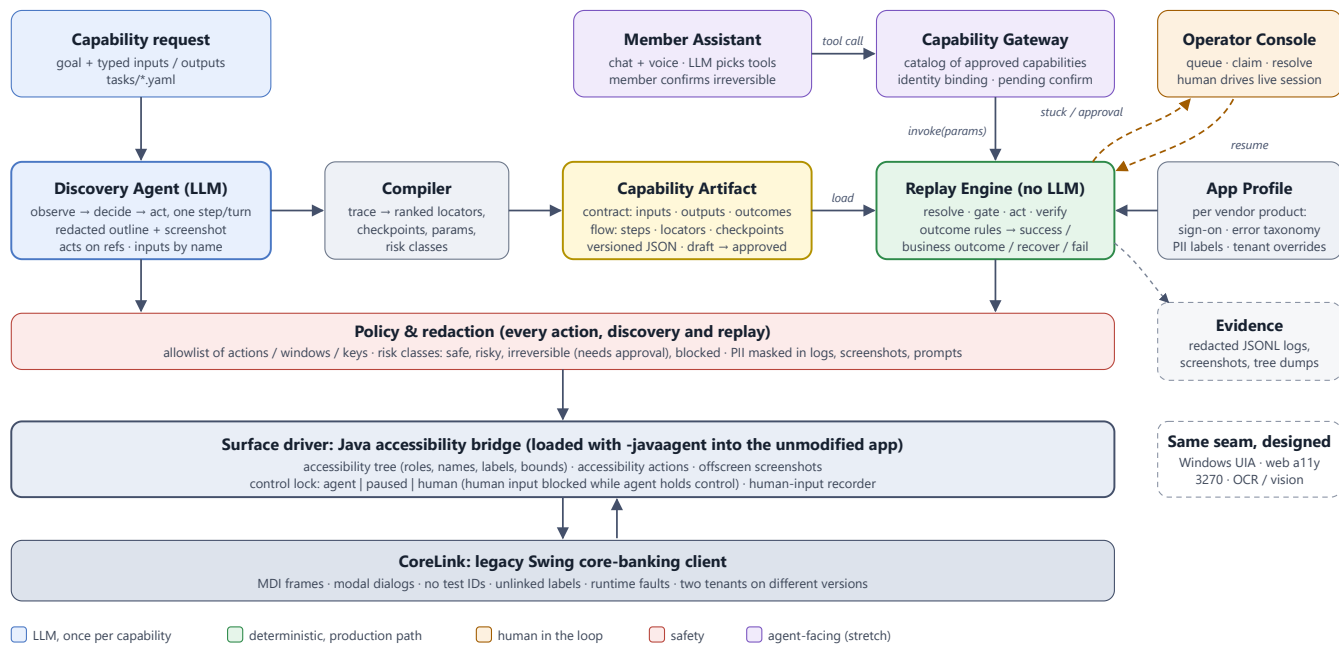
Discovery run	Planner	Success	Actions	Tokens in/out
discover-[CARD_OR_ACCOUNT]3ef0	llm:claude-opus-5	yes	5	59332/949
discover-[CARD_OR_ACCOUNT]ab13	llm:claude-opus-5	yes	10	163751/1827

Requirements coverage

Brief requirement	How it is met	Where in the repo
3.1 Goal + target in, observe, decide, act loop, stop conditions	A task YAML declares the goal, typed inputs, and outputs. The loop takes one action per turn. It stops on max steps, timeout, <code>finish</code> , or the human aborting.	<code>cua/agent/discover.py</code> , <code>tasks/</code>
3.1 Real UI, works without a clean DOM	Perception is the accessibility tree plus a screenshot of an unmodified Swing desktop app. Actions go through accessibility actions.	<code>bridge/</code> , <code>cua/surface/</code>
3.2 Typed, versioned, reviewable artifact	<code>cua.capability/v1</code> splits a contract (inputs, outputs, outcomes, risk) from a flow (steps, locators, checkpoints). It carries semver, draft or approved status, and provenance. JSON Schema is exported.	<code>cua/artifact.py</code> , <code>schemas/</code> , <code>capabilities/</code>
3.2 How targets are identified, and why that is robust	Five ranked strategies: a11y name, label, group, index path, then coordinates. Each records whether it was unique at record time. Replay reports fallbacks.	<code>cua/locate.py</code>
3.3 Replay with no LLM, stable targeting, checkpoints, outputs	Fixed start state, locator resolution with waits, checkpoint waits instead of sleeps, typed extraction, and success verification.	<code>cua/replay.py</code>

Brief requirement	How it is met	Where in the repo
3.3 Business outcome vs recoverable vs hard failure	Outcome rules come from the product profile. Results are <code>business_outcome</code> (with a code), <code>success</code> (with <code>recoveries[]</code>), or <code>failed</code> (step, expected, observed, evidence).	<code>configs/apps/corelink.yaml</code> , <code>cua/replay.py</code>
3.4 Configurable allowlist, risky actions handled conservatively	Actions, windows, and keys are allowlisted. Every target gets a risk class. Irreversible actions need an approval grant or a human.	<code>configs/policies/corelink.yaml</code> , <code>cua/policy.py</code>
3.4 No secrets or raw PII persisted	Secrets are referenced by name. Three-layer text redaction, screenshot blackout from accessibility bounds, and fingerprinted sensitive outputs. A unit test guards the artifacts.	<code>cua/redact.py</code> , <code>cua/secrets.py</code> , <code>tests/</code>
3.5 Structured log plus a richer failure signal	JSONL log of every decision (with the model's <code>why</code>), policy verdict, and checkpoint. Redacted screenshots and accessibility tree dumps on failure.	<code>cua/evidence.py</code> , <code>evidence/</code>
3.6 Detect and route with context	The intervention request carries the capability or goal, the step, the reason code, a redacted screenshot, and the session.	<code>cua/handoff.py</code>
3.6 Human takes the same live session, then hands back	Control lock (agent, paused, human) inside the app. Human input is blocked while the agent holds control and recorded while the human does. Resume re-verifies the checkpoint.	<code>bridge/</code> , <code>cua/handoff.py</code> , <code>cua/console.py</code>
3.7 Surface abstraction	<code>Observation</code> and <code>Node</code> plus a small verb set. Surface-specific code lives only in drivers.	<code>cua/surface/base.py</code> , REPORT §4
3.7 Multi-tenant reuse and drift	A product-level app profile with tenant overrides. The Heritage-recorded capability replays on Lakeshore (v7.6.2, renamed controls, extra dialog).	<code>configs/apps/corelink.yaml</code> , scenario 09
Stretch Agent-facing capability interface	A gateway exposes approved capabilities as typed tools. A member assistant (chat and voice) invokes them, with identity binding and member confirmation.	<code>cua/gateway.py</code> , <code>portal/</code>
Stretch Confidence and approval	Unattended replay requires <code>approved</code> . <code>cua approve</code> records the reviewer.	<code>cua/replay.py</code> , <code>cua/__main__.py</code>
Stretch Cross-tenant reuse with per-variant overrides	Tenant label and name aliases, plus extra outcome rules.	<code>configs/apps/corelink.yaml</code>

Architecture diagram



1. Architecture

Target choice. The proxy is CoreLink, a Java Swing desktop client I built to look like a legacy core-banking teller app. It uses an MDI desktop, modal option dialogs, upper-case status-bar errors, and no test IDs. On one form the labels are not wired to their fields. I chose a desktop app over a web demo because the brief says the common case has no clean DOM. A desktop app forces the perception and locator design to cope with that. I built it myself because every runtime condition the brief lists has to be reproducible on demand. Faults are injected through a properties file that the app re-reads on every action.

Layers. Each layer talks only to the one below it.

Layer	Responsibility	Code
Surface driver	Perceive (tree, screenshot), act (click, set text, select, key, close), control lock, input recorder	bridge/ (Java), cua/surface/
Policy + redaction	Allow, deny, or require approval for every action. Mask PII before anything leaves memory	cua/policy.py, cua/redact.py
Discovery	LLM observe, decide, act loop that records a trace	cua/agent/discover.py
Compiler	Trace to capability artifact, decoupled from the transcript	cua/agent/compile.py
Replay	Deterministic execution and outcome classification	cua/replay.py
Handoff	Pause, cede control, record the human, resume	cua/handoff.py, cua/console.py
Gateway	Catalog of approved capabilities as typed tools, with an invoke path	cua/gateway.py, portal/

Key decisions and trade-offs. - **An in-process accessibility bridge, not screenshots plus coordinates.** A - `javaagent` jar attaches to the unmodified app and reads `javax.accessibility`. That is the same role, name, state, and bounds data the Windows Java Access Bridge exposes to screen readers. Accessibility data is available for desktop apps (JAB, UI Automation) and for legacy web apps, so the design does not depend on a DOM. It is also far more stable than pixels. Actions go through accessibility actions rather than OS mouse events. They work while the desktop is locked and never collide with a person's input. The trade-off is that a vendor app might block agent injection. The fallback is the external Access Bridge, or UI Automation for non-Java apps, with the same Surface interface above it. - **The model points, the harness records.** The model acts only on element refs from the current outline. The compiler turns each targeted element into ranked locators computed from the live tree. The model never writes selectors, which keeps its hallucinations out of the artifact. - **Model is pluggable.** The planner interface has two real implementations. The default is Claude Opus 5, through the official SDK, with adaptive thinking. The other is Gemini, through google-genai. Both get the same redacted inputs and tool schema, both use a manual loop with one action per turn, and both log their reasoning summaries as evidence. Discovery is rare, so I favour the strongest model over cost there. Replay never calls a model. The planner used for each run is recorded in the artifact's provenance. - **One process, file-based intervention queue.** A desktop session is single-threaded by nature. The gateway serializes invocations with a lock. Queues and workers are deliberately not built; see Cuts.

2. Artifact schema

The schema is `cua.capability/v1`. It is Pydantic, exported as JSON Schema in `schemas/`. An example is `capabilities/corelink.member.get_share_savings_balance.v1.0.0.json`. The artifact is split into a **contract** and a **flow** because it has two readers.

Contract is what a calling agent reads: - `id`, `version` (semver), and `status`, which is draft, approved, or deprecated. - `title` and `description`. - `inputs`: typed `ParamSpec` with pattern or enum and a `sensitive` flag. - `outputs`: typed `OutputSpec` such as `currency`, with a `sensitive` flag. - `business_outcomes`: the codes the caller must handle, such as `MEMBER_NOT_FOUND`. - `risk`: `read_only`, `state_changing`, or `irreversible`.

`cua catalog` prints only this part.

Flow is what replay and a reviewer read: - `steps`: each step has an `intent` in plain English, an `action`, a `target`, a `value`, a `risk`, a list of `expect` checkpoints, and a `timeout_ms`. - `target = scope + strategies[]`. The scope names the window ("main" or a dialog-title regex) and the work-window container. Strategies are ranked and independent, and each records `unique_at_record`: 1. `ally`: role plus accessible name. This is the most semantic, and it is what a screen reader announces. 2. `label`: role plus visible label text, taken from the label relation or inferred from geometry (the nearest label on the same row, to the left). Legacy forms rarely wire labels, and a human finds the field this way too. 3. `group`: the nth control of a role inside a named panel, for example the table under "Share Accounts". 4. `path`: index path within the container. This is exact but breaks on layout change. 5. `coords`: a relative point in the container, the last resort. It also carries over to screenshot-only surfaces.

Replay reports which strategy matched. A fallback to strategy 3 shows drift before it becomes a failure. - `value`: `literal`, `param`, or `secret`. A secret is a name only. Credentials and example PII never appear in the artifact, and a unit test enforces that. - `extract`: value, name, or table cell (row match plus column), then a capture regex, then a parse type. - `success`: final checkpoints. - `outcome_rules`: capability-specific rules. Product-wide rules are inherited from the app profile. - `provenance`: discovering model, run id, goal, tenant, reviewer, and notes. Notes flag any step where a human intervened during discovery.

Why this shape. A capability is a function signature plus a reviewable procedure. Putting the contract first lets an agent choose a capability without parsing steps. Per-step `intent` and checkpoints let a reviewer approve it without watching a video. Declaring outcomes as data, mostly at the product level, means a new capability gets the full error taxonomy without new code.

3. Determinism & error handling

Determinism. - **Fixed start state.** Replay signs on if needed through the vendor-profile routine and closes every open work window before the first step. - **Ranked locators.** Each strategy must resolve to exactly one element. An ambiguous match falls through to the next strategy. - **No sleeps.** Each step waits for its checkpoints, bounded by `timeout_ms`: - `container_present` and `window_present` for new windows and frames; - `text_present` for status lines, generalized as `INQUIRY COMPLETE - \d+ SHARE\(\S\)`; - `element_value` for typed fields.

Text checkpoints count only text that is new since the action, so a stale status line can't satisfy one. The mock stamps status lines with a time, as many legacy cores do. - **Settle detection.** The UI counts as settled when the tree fingerprint stops changing. - **Input contract.** Inputs are validated against the contract before the UI is touched. A bad input is `rejected` / `INVALID_INPUT`, not a UI failure.

Error taxonomy. While resolving a target or waiting on a checkpoint, the engine evaluates outcome rules against modal dialogs and newly appeared text. Text we typed ourselves is excluded. First match wins, in this order: capability rules, then tenant extras, then product rules.

Class	Result status	Examples in CoreLink	Handling
Business outcome	<code>business_outcome</code> + <code>code</code>	<code>MEMBER_NOT_FOUND</code> , <code>MEMBER_RESTRICTED</code> , <code>MEMBER_CLOSED</code> , <code>VALIDATION_ERROR</code> , <code>PERMISSION_DENIED</code> , <code>OPERATOR_DECLINED</code>	Stop, return the code, the UI's own message, and a screenshot. This is not an error.
Recoverable	<code>success with recoveries[]</code>	<code>UNEXPECTED_DIALOG</code> (dismiss), <code>SLOW_RESPONSE</code> (keep waiting, bounded), <code>SESSION_EXPIRED</code> (dismiss, re-sign-on, restart once)	Bounded attempts. Every recovery is recorded in the result.
Hard failure	<code>failed</code> + <code>failure</code>	<code>HOST_ERROR</code> , <code>TARGET_NOT_FOUND</code> , <code>CHECKPOINT_NOT_MET</code> , <code>EXTRACTION_FAILED</code> , <code>POLICY_DENIED</code> , <code>RECOVERY_EXHAUSTED</code> , <code>HUMAN_TIMEOUT</code>	Stop. Report the step id and intent, what was expected, what was observed, a redacted screenshot, and a redacted tree snapshot.
Escalate	<code>continues</code> or <code>failed</code>	<code>SUPERVISOR_OVERRIDE_REQUIRED</code> , <code>SIGN_ON_REJECTED</code> , <code>tenant</code> <code>DISCLOSURE_CONFIRMATION</code>	Human handoff on the live session, then re-verify the checkpoint (section 5).

Sensitive outputs are returned to the caller but fingerprinted in the persisted `result.json`.

Evidence. `evidence/replays/summary.md` shows 15 scenarios, all passing: - the happy path and a second tenant; - not-found, restricted, and invalid input; - a popup, a slow host, and session expiry, all recovered; - a host error, which fails hard; - an irreversible post with a caller grant, with operator approval, and with no approval, which times out; - a validation error and a permission denial; - the supervisor handoff.

Drift (secondary). Locator fallbacks are logged per step. A tenant can override label and name aliases without re-recording. Structural drift that breaks every strategy fails with `TARGET_NOT_FOUND` and a full evidence bundle.

4. Heterogeneity & multi-tenant

Surface seam. Everything above `cua/surface/` sees only `Observation` (windows and nodes with role, name, value, label, group, and bounds) and a small verb set: click, set text, select, key, and close. The artifact stores semantic targets, not a technology's selectors.

Surface	Perceive	Act	What changes in the artifact
Java Swing (built)	In-process accessibility (JAB-equivalent)	Accessibility actions	Nothing
Other Windows desktop apps (.NET, PowerBuilder, Delphi)	UI Automation tree	UIA invoke and value patterns, with SendInput fallback	Nothing. Roles are mapped to the same vocabulary.
Legacy web (framesets, nested tables)	Browser accessibility tree across frames	CDP or Playwright	<code>scope.window</code> becomes a frame path
3270/5250 green screen	Screen buffer fields via an emulator API	Keystrokes	Label strategy maps to a protected field

Surface	Perceive	Act	What changes in the artifact
Citrix or pixels only	Screenshot, OCR, and a vision model fill Node with role guesses and bounds	Coordinates	label and coords strategies carry the load

Label inference and coordinate locators are already geometric, so they work unchanged on OCR output.

Multi-tenant reuse. Capabilities are scoped to a vendor product and a version range, not to a tenant. The app profile (`configs/apps/corelink.yaml`) holds what is shared by everyone running the product: - the sign-on routine; - PII labels; - the outcome taxonomy; - a `tenants:` block with small overrides: label and name aliases, extra outcome rules, and the product version.

The second tenant, Lakeshore, runs CoreLink 7.6.2. Its "Member #:" label is "Account No.:", its "Inquire" button is "Search", and it adds a disclosure dialog. The Heritage-recorded capability replays there unchanged, through the overrides (scenario 09). The resolution order is: capability, then tenant override, then product. That makes an override a small reviewable diff, not a fork.

Drift detection at scale. Four signals, in order of cost: 1. Locator fallback telemetry per capability per tenant. 2. The product version string, read at sign-on, checked against the capability's `versions` range. 3. Scheduled canary replays against a test member per tenant. 4. When a canary fails, a bounded re-discovery on that tenant diffs its artifact against the base. The compiler then proposes a tenant override, or a new capability version, for human review.

5. Escalation & handoff

Detecting "stuck." - **Discovery:** the model calls `ask_human`, or policy returns require-approval, or the step or time budget runs out. - **Replay:** an `escalate` rule matches, an irreversible step has no grant, or recovery is exhausted. Failures are reported and do not wait; see Cuts.

Control-transfer model. One holder at a time, enforced inside the app by the bridge:

```
agent --request--> paused --operator claims--> human --operator resolves--> agent
\----- timeout / abort -----> run ends
```

- `agent`: the bridge drops real keyboard and mouse input and shows an on-screen banner, so the operator cannot fight the automation. The automation's own input is marked as synthetic and passes through.
- `paused`: the automation stops touching the app and writes an intervention request. It carries the capability or goal, the step id and intent, the reason and rule code, a redacted screenshot, and the session address.
- `human`: the operator works in the same CoreLink window, with the same process, session, and signed-on operator. The bridge records their clicks, typing, and keys, with targets described semantically. Password fields are recorded as `[SECRET]`.
- **Resolve:** the operator chooses resume, approve, deny, or abort. The recorded actions are attached to the intervention and the run evidence, control returns to `agent`, and the engine re-verifies the current step's checkpoint. It never assumes the human left the app in the expected state.

Operator surface. The console (`cua console`) is intentionally minimal. It shows the queue, context, screenshot, a live session view, and the claim and resolve buttons. The CLI has the same verbs. The queue is a directory of JSON files, so a pager or ticketing hook is a small adapter.

Evidence. Scenario 15 shows a supervisor override. The handoff record lists five recorded human actions and a resume. For unattended evidence, `scripts/sim_operator.py` stands in for the person. It injects real input events so the recorder sees them. Its interventions are labeled `sim-supervisor` so they are never mistaken for a real human.

6. Safety

Guardrail model. Enforced before every action, in discovery and in replay alike: - an **allowlist** of action verbs, window titles, and keys; - **risk rules** that classify each target as safe, risky, irreversible, or blocked.

Irreversible means approval required. I chose that over a hard block, because the system must do real work, and over flag-only, because a posted transaction cannot be recalled. Approval comes from one of three places: - a human operator; - a per-invocation grant from the caller, for an approved capability; - in the member assistant, the member pressing Confirm in the UI. The model cannot confirm for them.

One approval covers one transaction per run. Draft capabilities can't run unattended.

Identity binding. The gateway removes caller-bound parameters, such as the signed-in member's own number, from the tool schema. Bound values override anything the model passes. A prompt-injected "check member 23456" has nothing to act on.

Data handling. Redaction happens before anything leaves memory: logs, evidence, intervention records, and prompts to the discovery model. Three layers work together: - patterns for SSNs, dates, card and account numbers, and amounts; - label rules from the app profile, such as the value next to "SSN:"; - run-scoped literals for inputs and outputs marked sensitive.

Screenshots are redacted by blacking out the accessibility bounds of sensitive nodes. The model never sees raw PII or credentials. Secrets are referenced by name and resolved from `env/.env`, standing in for a vault. Sensitive outputs are persisted only as fingerprints. A unit test fails if an artifact contains a sample member number, an SSN prefix, a balance, or a credential.

Limits. - Label and pattern redaction can miss PII in free text, such as a note field. Production needs a classifier and allowlisted fields. - The member assistant sends the member's own balance to the model to phrase the answer. - The live console view is unredacted, which assumes an authorized operator. - The policy trusts window titles, and a hostile app could spoof them. - The bridge listens on localhost without authentication. - Approval grants are not yet cryptographically bound to the reviewed artifact hash.

7. Cuts

Mocked or stubbed, on purpose: - **Target app.** CoreLink is a proxy. No real bank system was used. - **Operator console.** It is minimal, with no browser-based remote input. The operator uses the real window. - **Simulated operator.** It is used for unattended evidence runs only. - **Member sign-in.** The portal login is a mock. - **Secret store.** It reads `env/.env` instead of a vault. - **Surfaces.** Only one is implemented. UIA, web, and terminal are designed, not built.

Not built: - queues, workers, and multi-session scaling (per the brief); - tenant-level scheduling; - artifact storage beyond files plus git; - authentication on the bridge, console, and gateway; - LLM-assisted fallback during replay; - multi-run stability scoring (the scenario matrix is a start).

Next, in order: 1. **Bounded assisted fallback.** On `TARGET_NOT_FOUND`, allow one policy-checked LLM re-grounding of that single step, then record the proposed locator as a draft override. 2. **Canary replays and drift diffing**, from section 4. 3. **A UI Automation surface**, to prove the seam on a second desktop technology. 4. **Artifact signing.** Approval binds to the hash of the reviewed artifact. 5. **Hardening.** Authentication and mTLS on the bridge, and a PII classifier on free text.

Appendix A. Demo commands

```
.\scripts\build.ps1 # setup: venv, portable JDK, Java builds
python -m cua discover --task tasks\get_savings_balance.yaml # 1. LLM discovery, then artifact
python -m cua catalog # 2. contract view
python -m cua approve corelink.member.get_share_savings_balance --reviewer <name>
python -m cua replay corelink.member.get_share_savings_balance -p member_number=12345 # 3. replay
python -m cua replay corelink.member.get_share_savings_balance -p member_number=99999 # 4. not found
python -m cua replay ... --fault session_expired=true | --fault app_error=true # recover /
fail
python -m cua console # 5. operator console (handoff)
python scripts\scenarios.py --out evidence\replays # 6. full scenario matrix
python -m portal.app --allow-draft # 7. member assistant, chat + voice
```

Appendix B. Replay scenario results

Scenario	Tenant	Injected	Result	Code	Recoveries	Pass
01-balance-happy-path	heritage	-	success	success	-	yes
02-balance-member-not-found	heritage	-	business_outcome	MEMBER_NOT_FOUND	-	yes
03-balance-invalid-input	heritage	-	rejected	INVALID_INPUT	-	yes
04-balance-member-restricted	heritage	-	business_outcome	MEMBER_RESTRICTED	-	yes
05-balance-unexpected-popup	heritage	popup=true	success	success	UNEXPECTED_DIALOG	yes
06-balance-slow-host	heritage	slow_ms=4500	success	success	SLOW_RESPONSE	yes
07-balance-session-expired	heritage	session_expired=true	success	success	SESSION_EXPIRED	yes
08-balance-host-error	heritage	app_error=true	failed	HOST_ERROR	-	yes
09-balance-second-tenant	lakeshore	-	success	success	-	yes
10-open-account-approved	heritage	-	success	success	-	yes

Scenario	Tenant	Injected	Result	Code	Recoveries	Pass
11-open-account-below-minimum	heritage	-	business_outcome	VALIDATION_ERROR	-	yes
12-open-account-permission-denied	heritage	-	business_outcome	PERMISSION_DENIED	-	yes
13-open-account-no-approval-grant	heritage	-	failed	HUMAN_TIMEOUT	-	yes
14-open-account-operator-approves	heritage	-	success	success	-	yes
15-open-account-supervisor-handoff	heritage	-	success	success	-	yes

Each scenario runs in a fresh app instance. The evidence for each is in `evidence/replays/<scenario>/`. Scenarios 14 and 15 use a clearly labeled simulated operator in place of a person at the keyboard.

Appendix C. Artifact excerpt

Contract (what a calling agent sees)

```
{
  "id": "corelink.member.get_share_savings_balance",
  "version": "1.0.0",
  "status": "draft",
  "risk": "read_only",
  "inputs": [
    {
      "name": "member_number",
      "type": "string",
      "description": "5-digit CoreLink member number",
      "pattern": "^\\d{5}$",
      "required": true,
      "sensitive": true
    }
  ],
  "outputs": [
    {
      "name": "savings_balance",
      "type": "currency",
      "description": "Current Share Savings balance in USD",
      "sensitive": true
    }
  ],
  "business_outcomes": [
    "MEMBER_NOT_FOUND",
    "MEMBER_RESTRICTED",
    "MEMBER_CLOSED",
    "PERMISSION_DENIED",
    "VALIDATION_ERROR"
  ]
}
```

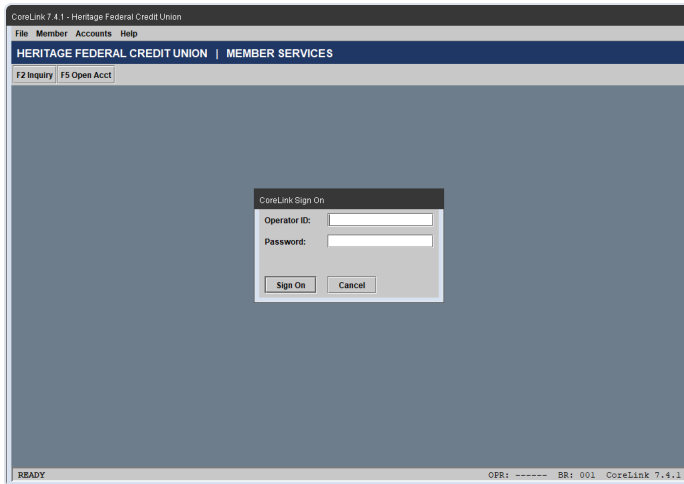
One flow step (what replay executes)

```

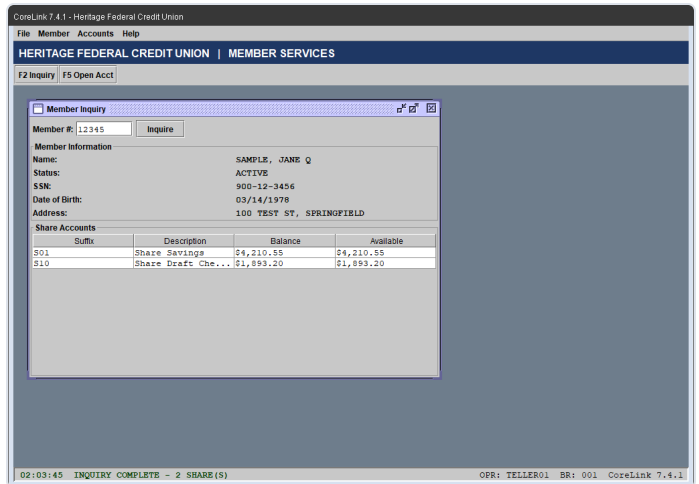
{
  "id": "s2",
  "intent": "Enter the member number to search.",
  "action": "set_text",
  "target": {
    "description": "text 'Member #:' in 'Member Inquiry'",
    "scope": {
      "window": "main",
      "container": "^Member Inquiry$"
    },
  },
  "strategies": [
    {
      "kind": "ally",
      "role": "text",
      "name": "Member #:",
      "unique_at_record": true
    },
    {
      "kind": "label",
      "role": "text",
      "label": "Member #:",
      "unique_at_record": true
    },
    {
      "kind": "path",
      "role": "text",
      "path": "0.1.0.0.1",
      "unique_at_record": true
    },
    {
      "kind": "coords",
      "rel_x": 0.1871,
      "rel_y": 0.1071,
      "unique_at_record": true
    }
  ],
  "value": {
    "source": "param",
    "param": "member_number"
  },
  "risk": "safe",
  "expect": [
    {
      "kind": "element_value",
      "pattern": ".+",
      "description": "field holds the entered value"
    }
  ],
  "timeout_ms": 8000
}

```

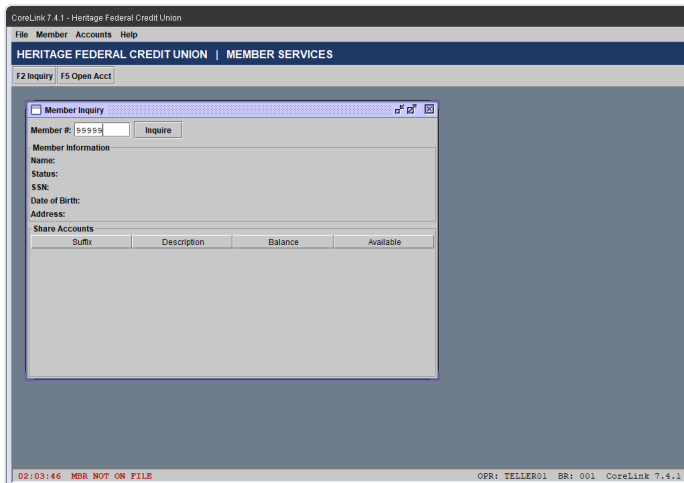
Appendix D. Screens



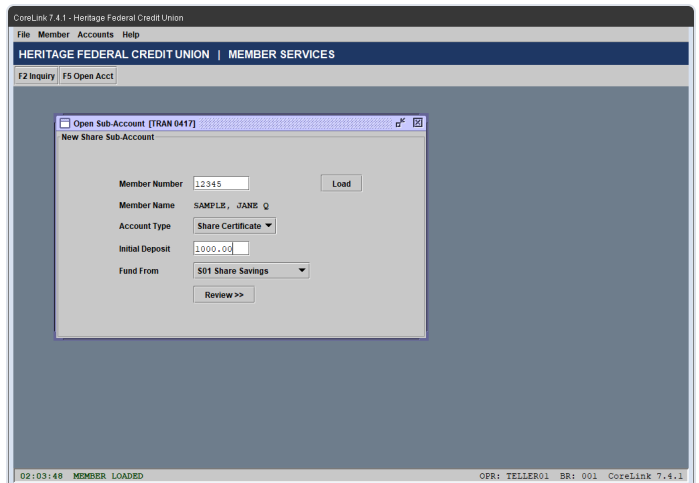
Sign-on. Credentials are resolved from the secret store by name.



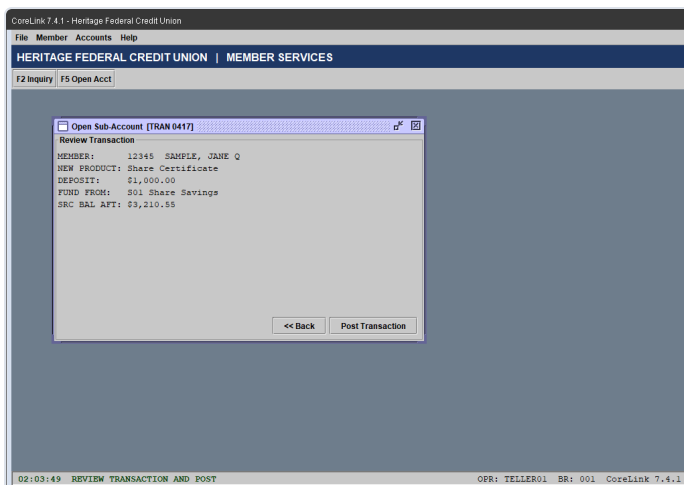
Member Inquiry: the flow the first capability automates.



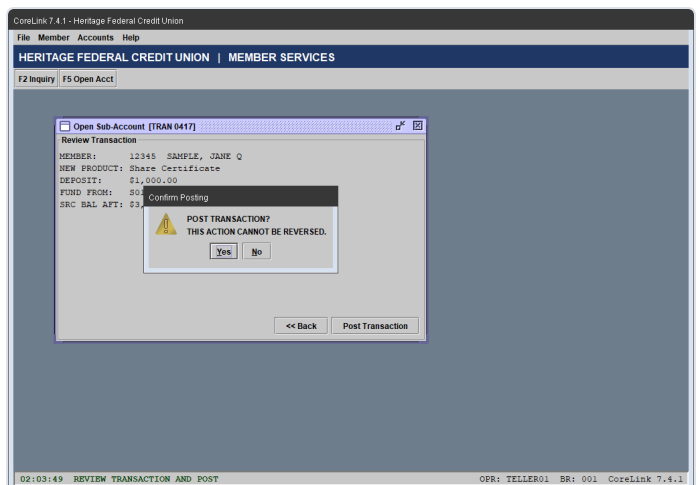
Business outcome: MBR NOT ON FILE becomes MEMBER_NOT_FOUND.



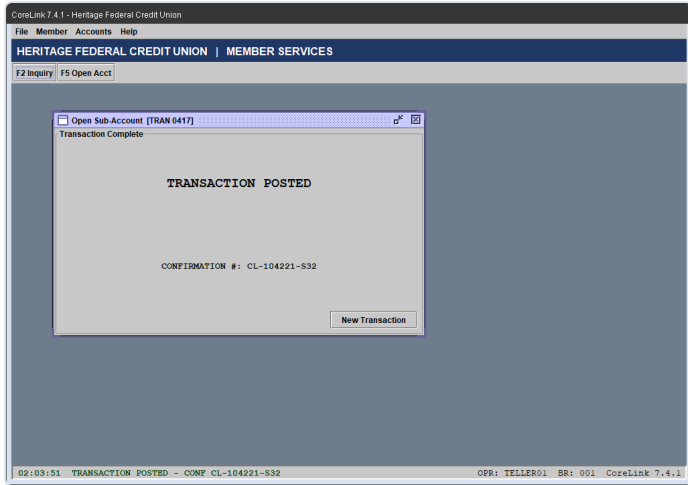
The labels are not linked to their fields. Replay locates them by label geometry.



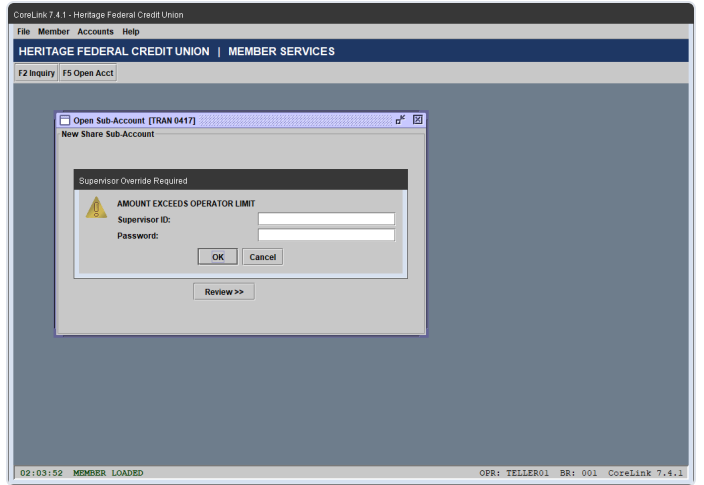
The review screen, before the irreversible post.



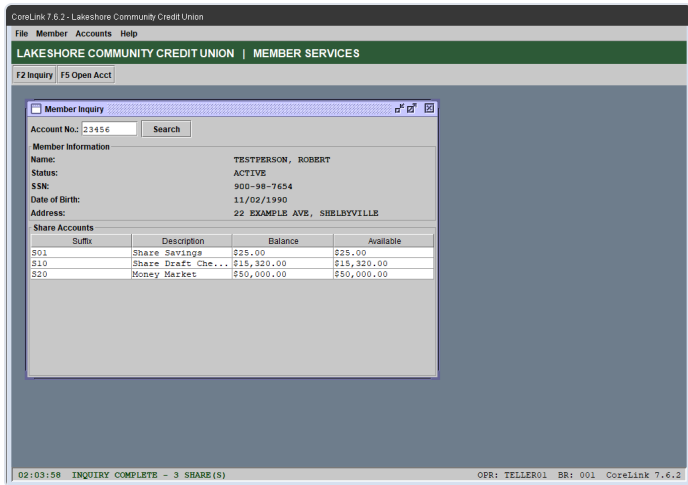
An irreversible confirmation. Policy requires approval.



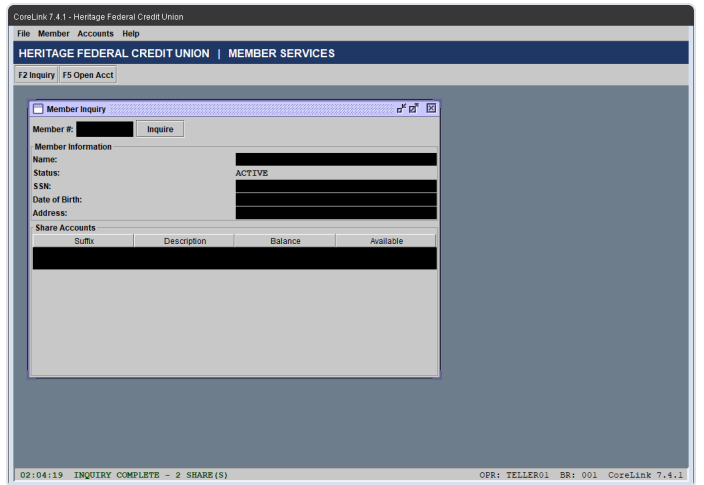
The output: a confirmation number extracted with a capture pattern.



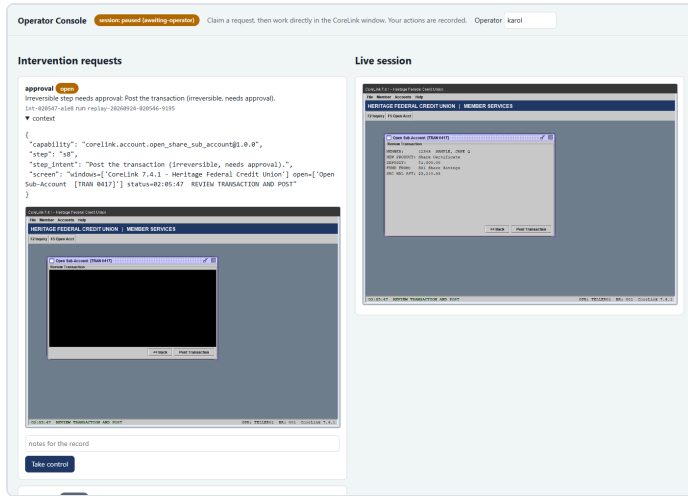
Escalation: a supervisor override is handed to a human.



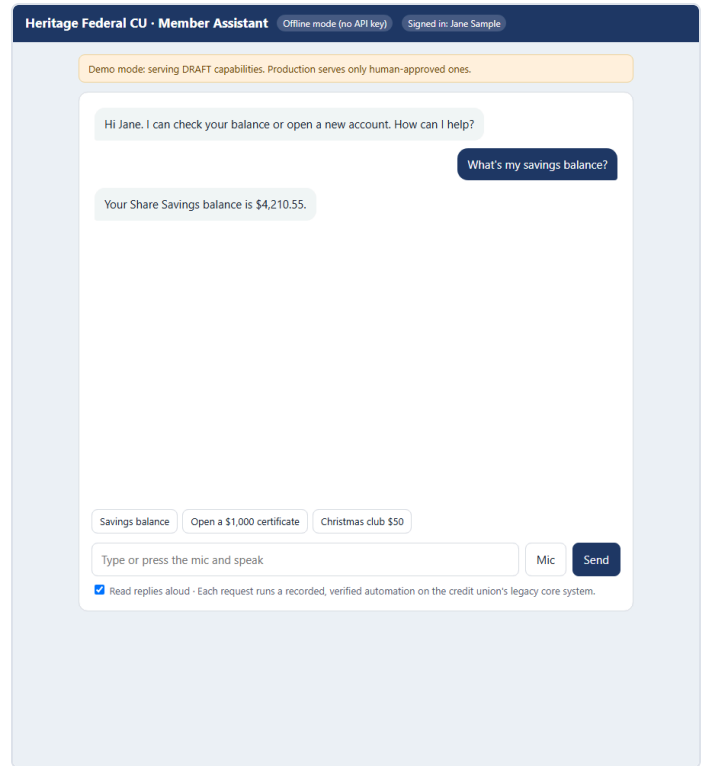
Second tenant (v7.6.2, renamed controls). The same capability replays through overrides.



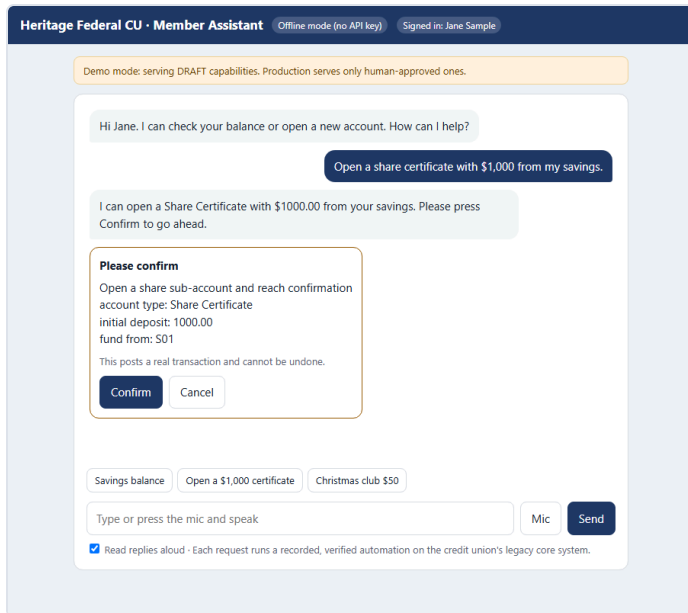
Evidence screenshot, redacted using accessibility bounds.



Operator console: the queue, context, redacted snapshot, and live session.



Member assistant answering from a deterministic replay.



The member must confirm an irreversible action. The model cannot.